

Data Structures Using C	
BVNSD 2.1	
Credits:4	40 Hours
Introduction:	
Data structure is an implementation of C,Java,Python and other language in which knowledge of program logic development and Algorithm of the problem are given.The concept of static programming and Dynamic programming both are teach.	
Course Objectives:	
The main objective of this course is to develop the Program development logic and various technique to solve the scientific as well as commercial problem	
Unit 1:	10 Hours
Representation of Single and Multidimensional Arrays; SparseArrays– Lower and Upper Triangular Matrices and Tridiagonal Space Matrices with Vector Representation.	
Unit 2:	10 Hours
Stack, Queues, Singly Linked List, Doubly Linked List, Circular Linked Lists, Implementing Pointers and Objects, Representing Rooted Trees.	
Unit 3:	10 Hours
Heap Sort, Quick Sort, Counting Sort, Radix Sort, Bucket Sort, Median and Order Statistics.	
Unit 4:	10 Hours
Introduction and Terminology; Traversal of Binary Trees; Recursive Algorithms for Tree Operations such As Traversal, Insertion, Deletion; Binary Search Tree; B-Tree; Indexing with Binary Search Trees. Direct Address Tables, Hash Tables, Hash Functions, Open Addressing.	
Course Outcomes(COs):	
CO1: Analyze algorithms and algorithm correctness.	
CO2: Implement searching and sorting techniques.	
CO3: Demonstrate stack, queue and linked list operation.	
CO4: Understand the concepts of tree and graphs.	
Text Books:	
1. Fundamentals of Data structures Using C: Horwitz and Sahni (Silicon Press) 2. Data Structures & Algorithms: R.S.Salaria (KhannaPublishers) 3. Data Structures using C and C++: Langsam, Augenstein, and Tenenbaum (PHI) 4. Introduction to Algorithms: Thomas H. Coreman, Charles E.Leiserson and Ronald L.Rivest (MIT Press)	

Lab on Data Structures and C Programming (Practical Paper)**BVNSD 2.2****Credits:4****40 Hours****Introduction:**

Data structure is an implementation of C, Java, Python and other language in which knowledge of program logic development and Algorithm of the problem are given. The concept of static programming and Dynamic programming both are teach.

Course Objectives:

The main objective of this course is to develop the Program development logic and various technique to solve the scientific as well as commercial problem

Practical:**40 Hours**

1. Write a program to manipulate stack using push, pop, display and search operation.
2. Write a program to convert infix expression into postfix and postfix into infix.
3. Write a program to implement queue using Insert, Delete, search operation.
4. Write a program to implement circular queue using Insert, Delete, search operation.
5. Write a program to implement dequeue using Insert, Delete, search operation.
6. Write a program to implement Linear Linked List. Using inset, delete, add at beg, append, display and count function.
7. Write a program to implement circular Linked List. Using inset, delete, add at beg, append, display and count function.
8. Write a program to implement Doubly Linked list. Using inset, delete, add at beg, append, display and count function.
9. Write a program to construct the Binary tree and also traverse the binary tree in in order, pre order, post order.
10. Write program to perform following sorting.
 - a) bubble sorting
 - b) Quick sorting
 - c) Selection sorting
 - d) Heap sorting
 - e) Merge Sorting.
11. Perform the binary search.

12. Perform the linear search.
13. Write a program to implement DFS, BFS.
14. Write a program to implement direct search.
15. Write a program to implement prims algo.

Course Outcomes(COs):

CO1: Analyze and write code linked list programs.

CO2: Implement code searching and sorting techniques.

CO3: Demonstrate & code stack, queue and tree list operation.

CO4: Implement code graphs concepts

Text Books:

5. Fundamentals of Data structures Using C: Horwitz and Sahni (Silicon Press)
6. Data Structures & Algorithms: R.S.Salaria (KhannaPublishers)
7. Data Structures using C and C++: Langsam, Augenstein, and Tenenbaum (PHI)
8. Introduction to Algorithms: Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest (MIT Press)

Design And Analysis Of Algorithm	
BVNSD 4.2	
Credits:4	40 Hours
Introduction:	
Design and analysis of algorithms is the process of finding the computational complexity of algorithms – the amount of time, storage, or other resources needed to execute them.	
Course Objectives:	
This course will cover basic concepts in the design and analysis of algorithms. Asymptotic complexity, $O()$ notation Sorting and search. Algorithms on graphs: exploration, connectivity, shortest paths, directed acyclic graphs, spanning trees. Design techniques: divide and conquer, greedy, dynamic programming. Data structures: heaps, union of disjoint sets, search trees Intractability.	
Note: 3 Credits for Theory and 1Credit for Practical	
Unit 1:	7 Hours
Introduction: Algorithms, Analyzing Algorithms, Complexity of Algorithms, Growth of Functions, Performance Measurements, Sorting and Order Statistics - Shell Sort, Quick Sort, Merge Sort, Heap Sort, Comparison of Sorting Algorithms, Sorting in Linear Time.	
Unit 2:	7 Hours
Advanced Data Structures: Red-Black Trees, B – Trees, Binomial Heaps, Fibonacci Heaps, Tries, Skip List.	
Unit 3:	7 Hours
Divide and Conquer with Examples Such as Sorting, Matrix Multiplication, Convex Hull and Searching. Greedy Methods with Examples Such as Optimal Reliability Allocation, Knapsack, Minimum Spanning Trees – Prim’s and Kruskal’s Algorithms, Single Source Shortest Paths - Dijkstra’s and Bellman Ford Algorithms.	
Unit 4:	7 Hours
Dynamic Programming with Examples Such as Knapsack. All Pair Shortest Paths – Warshal’s and Floyd’s Algorithms, Resource Allocation Problem. Backtracking, Branch and Bound with Examples Such as Travelling Salesman Problem, Graph Coloring, n-Queen Problem, Hamiltonian Cycles and Sum of Subsets.	
Practical:	12 Hours
1. Program for Recursive Binary & Linear Search. 2. Program for Heap Sort. 3. Program for Merge Sort. 4. Program for Selection Sort. 5. Program for Insertion Sort. 6. Program for Quick Sort. 7. Knapsack Problem using Greedy Solution 8. Perform Travelling Salesman Problem 9. Find Minimum Spanning Tree using Kruskal’s Algorithm 10. Implement N Queen Problem using Backtracking 11. Implement , the 0/1 Knapsack problem using (a) Dynamic Programming method (b) Greedy method. 12. From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm. 13. Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.	

14. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.
15. Write programs to Implement All-Pairs Shortest Paths problem using Floyd's algorithm.
16. Write programs to Implement Travelling Sales Person problem using Dynamic programming.
17. Design and implement to find a subset of a given set $S = \{S_1, S_2, \dots, S_n\}$ of n positive integers whose SUM is equal to a given positive integer d . For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$, there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. Display a suitable message, if the given problem instance doesn't have a solution.
18. Design and implement to find all Hamiltonian Cycles in a connected undirected Graph G of n vertices using backtracking principle.

Course Outcomes(COs):

CO1: Implement new algorithms, prove them correct, and analyze their asymptotic and absolute runtime and memory demands.

CO2: Understand the Advanced Data Structures

CO3: Implement the Divide and Conquer, Greedy Methods and Trees.

CO4: Apply the Dynamic Programming with examples.

Text Books:

1. Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest, "Introduction to Algorithms", Printice Hall of India.
2. E. Horowitz & S Sahni, "Fundamentals of Computer Algorithms",
3. Aho, Hopcraft, Ullman, "The Design and Analysis of Computer Algorithms" Pearson Education, 2008.
4. LEE "Design & Analysis of Algorithms (POD)", McGraw Hill
4. Richard E. Neapolitan "Foundations of Algorithms" Jones & Bartlett Learning
5. Jon Kleinberg and Éva Tardos, Algorithm Design, Pearson, 2005.